МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

Кафедра вычислительной математики

МЕТОДЫ РЕШИЯ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ

Студент: Король Денис Евгеньевич
3 курс 8 группа

Преподаватель: Мандрик Павел Алексеевич

Минск, 2026

Введение

Программа реализована на языке Java (версия 24, non-LTS) с использованием стандартной библиотеки java.util и java.lang.Math. Компиляция и запуск выполнялись в среде IntelliJ IDEA 2023.2 на платформе OpenJDK 24.0.2. Исходный код программ приведён в приложениях А–В.

Исходное ДУ и его преобразование

Исходная система выглядит так:

$$\begin{cases} u''(x) - xu'(x) + u(x) = -x^2 + 4, 0 < x < 1 \\ u(0) = 2 \\ u(1) = 3 \end{cases} \quad (1)$$

Точное решение:

$$u(x) = x^2 + 2 \quad (2)$$

Отрезок [0,1] разбивается на n равных частей с шагом $h = 1/n$. Вводятся узлы сетки:

$$x_i = ih, i = 0,1, \dots, n \quad (3)$$

Значения искомой функции в узлах обозначаются $y_i \approx u(x_i)$. Производные аппроксимируются конечными разностями:

$$u'(x_i) \approx \frac{y_{i+1} - y_i}{h}, i = 0, \dots, n-1 \quad (4)$$

$$u''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}, i = 1, \dots, n-1 \quad (5)$$

Подставляя (4) и (5) в уравнение (1) в узле $x_i = ih$, умножая обе части на h^2 получаем уравнение:

$$y_{i-1} + (h^2 + hx_i - 2)y_i + (1 - hx_i)y_{i+1} = \left(4 - x_i^2\right)h^2, i = 1, \dots, n-1 \quad (6)$$

где

$$a_i = 1, i = 1, \dots, n-1$$

$$c_i = (h^2 + hx_i - 2), i = 1, \dots, n-1$$

$$b_i = (1 - hx_i), i = 1, \dots, n-1$$

$$d_i = \left(4 - x_i^2\right)h^2, i = 1, \dots, n-1 \qquad (7)$$

Краевые условия (1) на сетке записываются как

$$y_0 = 2, y_n = 3 \qquad (8)$$

Уравнение (6) выписывается только для внутренних узлов $i = 1, \dots, n-1$. Значения y_0 и y_n известны и переносятся в правую часть первого и последнего уравнений соответственно. В результате получается система линейных алгебраических уравнений (СЛАУ) порядка $n-1$ с трёхдиагональной матрицей:

$$\begin{pmatrix} c_1 & b_1 & 0 & \cdots & 0 \\ a_2 & c_2 & b_2 & \cdots & 0 \\ 0 & a_3 & c_3 & \ddots & \vdots \\ \vdots & & \ddots & \ddots & b_{n-2} \\ 0 & \cdots & \cdots & a_{n-1} & c_{n-1} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-2} \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} d_1 - 2a_1 \\ d_2 \\ \vdots \\ d_{n-2} \\ d_{n-1} - 3b_{n-1} \end{pmatrix} \qquad (9)$$

Подсчет всех коэффициентов и ответов происходит в функции completeTask3, она представлена в приложении А, а сама матрица создается благодаря классу TridiagonalMatrix из приложения Б.

Метод прогонки

Сначала считаем прогоночные коэффициенты:

$$\alpha_1 = \frac{-b_1}{c_1}, \beta_1 = \frac{d_1}{c_1}$$

$$\alpha_{i+1} = \frac{-b_i}{c_i + \alpha_i a_i}, i = 1, \dots, n-2$$

$$\beta_{i+1} = \frac{d_i - \beta_i a_i}{c_i + \alpha_i a_i}, \ i = 1, \dots, n-1 \qquad (10)$$

Потом вычисляем сами корни:

$$y_{n-1} = \beta_n$$
$$y_i = \alpha_{i+1} y_{i+1} + \beta_{i+1}, i = n-2, \dots, 1. \qquad (11)$$

Реализация метода выполнена в классе TridiagonalMatrixMethod и её код указан в приложении В.

Результат

Результаты найденных решений СЛАУ для $n = 3, 5, 7$

Скриншот для 3:

```
Enter Number: 3
Y:           2,00000000e+00      2,09060847e+00      2,41732804e+00      3,00000000e+00

exact Y:     2,00000000e+00      2,11111111e+00      2,44444444e+00      3,00000000e+00

Difference:  0,00000000e+00      -2,05026455e-02     -2,71164021e-02     0,00000000e+00
```

Скриншот для 5:

```
Enter Number: 5
Y:           2,00000000e+00      2,03242045e+00      2,14650756e+00      2,34414540e+00      2,62763633e+00      3,00000000e+00

exact Y:     2,00000000e+00      2,04000000e+00      2,16000000e+00      2,36000000e+00      2,64000000e+00      3,00000000e+00

Difference:  0,00000000e+00      -7,57955330e-03     -1,34924399e-02     -1,58546019e-02     -1,23636677e-02     0,00000000e+00
```

Скриншот для 7:

```
Enter Number: 7
Y:           2,00000000e+00      2,01653973e+00      2,07432097e+00      2,17379600e+00      2,31546573e+00      2,49990617e+00      2,72780326e+00      3,00000000

exact Y:     2,00000000e+00      2,02040816e+00      2,08163265e+00      2,18367347e+00      2,32653061e+00      2,51020408e+00      2,73469388e+00      3,00000000

Difference:  0,00000000e+00      -3,86842880e-03     -7,31168754e-03     -9,87746762e-03     -1,10648865e-02     -1,02979083e-02     -6,89062074e-03     0,00000000
```

ПРИЛОЖЕНИЕ А

Подсчет коэффициентов и ответов

```
public static void completeTask3() {
    int n = ConsoleIO.enterNum();
    int size = n - 1;

    double[] a = new double[size];
    double[] c = new double[size];
    double[] b = new double[size];
    double[] d = new double[size];
    double[] y;

    double h = 1.0 / n;
    for (int j = 0; j < size; j++) {
        int i = j + 1;
        a[j] = 1.0;
        c[j] = (1 + i) * h * h - 2;
        b[j] = 1.0 - i * h * h;
        d[j] = (4.0 - (i * i * h * h)) * (h * h);
    }

    d[0] -= a[0] * 2;
    a[0] = 0;

    d[size - 1] -= b[size - 1] * 3;
    b[size - 1] = 0;

    TridiagonalMatrix TridiagonalMatrix = new TridiagonalMatrix(a, c, b);
    y = TridiagonalMatrixMethod.solve(TridiagonalMatrix.getMatrix(), d);

    double[] yFull = new double[n + 1];
    yFull[0] = 2;
    yFull[n] = 3;
    System.arraycopy(y, 0, yFull, 1, size);

    double[] exactY = new double[n + 1];
    for (int i = 0; i < n + 1; i++) {
        exactY[i] = i * i * h * h + 2;
    }

    VectorUtils.showVector("Y: ", yFull);
    VectorUtils.showVector("exact Y: ", exactY);
    VectorUtils.showDifference("Difference: ", yFull, exactY);

}
```

ПРИЛОЖЕНИЕ Б

Создание трёх-диагональной матрицы

```
public class TridiagonalMatrix {
    private final int size;
    private final double[][] matrix;

    public TridiagonalMatrix(double[] a, double[] c, double[] b) {
        if (a.length != c.length || b.length != c.length) {
            throw new IllegalArgumentException("A, B and C sizes must be
equal");
        }
        size = c.length;
        matrix = new double[size][size];

        for (int i = 0; i < size; i++) {
            matrix[i][i] = c[i];
            if (i != 0) {
                matrix[i][i - 1] = a[i];
            }
            if (i != size - 1) {
                matrix[i][i + 1] = b[i];
            }
        }
    }

    public double[][] getMatrix() {
        double[][] copy = new double[size][size];
        for (int i = 0; i < size; i++) {
            copy[i] = matrix[i].clone();
        }
        return copy;
    }

    public int getSize() {
        return size;
    }

    public void showMatrix02F() {
        for (double[] i : matrix) {
            for (double j : i ) {
                System.out.printf("%-10.2f", j);
            }
            System.out.println();
        }
        System.out.println();
    }
}
```

ПРИЛОЖЕНИЕ В

Метод прогонки

```
public class TridiagonalMatrixMethod {

    public static double[] solve(double[][] matrix, double[] rhs) {
        validate(matrix, rhs);

        int size = matrix.length;
        double[][] a = new double[size][size];
        for (int i = 0; i < size; i++) {
            a[i] = matrix[i].clone();
        }

        double[] x = new double[size];
        double[] aMas = new double[size + 1];
        double[] bMas = new double[size + 1];

        aMas[1] = -1 * a[0][1] / a[0][0];
        bMas[1] = rhs[0] / a[0][0];
        for (int i = 1; i < size - 1; i++) {
            aMas[i + 1] = -1 * a[i][i + 1] / (a[i][i] + a[i][i - 1] * aMas[i]);
            bMas[i + 1] = (rhs[i] - a[i][i - 1] * bMas[i]) / (a[i][i] + a[i][i -
1] * aMas[i]);
        }
        bMas[size] = (rhs[size - 1] - a[size - 1][size - 2] * bMas[size - 1])
                / (a[size - 1][size - 1] + a[size - 1][size - 2] * aMas[size -
1]);

        x[size - 1] = bMas[size];
        for (int i = size - 2; i >= 0; i--) {
            x[i] = x[i + 1] * aMas[i + 1] + bMas[i + 1];
        }
        return x;
    }
    private static void validate(double[][] matrix, double[] rhs) {
        if (matrix == null || rhs == null) {
            throw new IllegalArgumentException("RHS or/and matrix must not be
null");
        }

        int n = matrix.length;
        if (n == 0) {
            throw new IllegalArgumentException("Matrix must not be empty");
        }

        if (rhs.length != n) {
            throw new IllegalArgumentException("RHS and matrix sizes must be
equal");
        }

        for (double[] doubles : matrix) {
            if (doubles.length != n) {
                throw new IllegalArgumentException("Matrix must be square");
            }
        }
    }
}
```